National University of Computer & Emerging Sciences

Lecture 24

Hardware Solutions

CS Problem – S/W based solutions

- Bakery and Peterson Solutions solve the critical section problem – in software

- In order to implement an efficient lock why shouldn't we use algorithms such as Peterson or Bakery?

Why does it fail then?

- Not because of our logic, but because of wrong
- assumptions about the real world.

- Mutual exclusion depends on the order of steps in the software based algorithms.

- However, modern Compilers often reorder instructions to enhance performance

Hardware Approach- CS problem

1. Disabling Interrupts
2. Test Set and Lock
3. Compare and swap

Disabling Interrupts

- Solution: A Process
  - Disable interrupts before it enters its critical section
  - Enable interrupts after it leaves its critical section
- CPU will be unable to switch a process while it is in its critical section
- Guarantees that the process can use the shared variable without another process accessing it
- Disadvantage:
  - Unwise to give user processes this much power
  - The computer will not be able to service useful interrupts
  - The process may never enable interrupts, thus (effectively) crashing the system
- However, the kernel itself can disable the interrupts

On a uniprocessor

- On a uniprocessor, the only thing that will make multiple instruction sequences not atomic is interrupts.
- So, if want to do a critical section, turn off interrupts before the critical section and turn on interrupts after the critical section.
- Guaranteed atomicity. It is also fairly efficient.
- Early versions of Unix did this.

Mutual Exclusion: Hardware Support

– A process runs until it invokes an operating-system service or until it is interrupted

– Disabling interrupts guarantees mutual exclusion

```
while(true){
    /* disable interrupts */
        critical section
    /* enable interrupts */
        remainder section
}
```

Why not just use turning off interrupts?

- main disadvantages:
  - can't use in a multiprocessor, and
  - Unwise to give user processes this much power
  - The computer will not be able to service useful interrupts
  - The process may never enable interrupts, thus (effectively) crashing the system

FLAG = FALSE

Software Lock

The solution will work, if Testing and Setting the Flag are atomic

Process 1

```
1.while (FLAG == FALSE);
2.FLAG = FALSE;
3.critical_section();
4.FLAG = TRUE;
5.noncritical_section();
```

Process 2

```
1.while (FLAG == FALSE);
2.FLAG = FALSE;
3.critical_section();
```

Timeout

No two processes may be simultaneously inside their critical sections

Hardware Lock

- Some instruction sets assist us in implementing mutual exclusion on multiple processors

```
boolean TestAndSet (boolean *target)
  {
      boolean rv = *target;
      *target = TRUE;
      return rv:
  }
```

Guaranteed to be Atomic

TSL is "Atomic"

- Atomic => Uninterruptible
- No other process can access the memory until TSL is complete
- The CPU executing the TSL instruction locks the memory bus
- Thus, prohibits others CPUs from accessing the memory
- It's been used for things such as locking mailbox files.

TSL (test-and-set LOCK)

- is an **instruction** used to write 1 (**set**) to a memory location
- return its old value as a single atomic (i.e., non-interruptible) operation.

- The calling process obtains the **lock**

  if the old value was 0,
- otherwise the while-loop spins waiting to acquire the **lock**.

Mutual Exclusion Using TSL

- Shared boolean variable lock., initialized to false.
- Solution:

```
do {
        while ( TestAndSet (&lock ))
                ;   // do nothing

                //    critical section

        lock = FALSE;

                //      remainder section

} while (TRUE);
```

```
repeat

        while Test-and-Set(lock) do no-op;
        critical section
        lock := FALSE;
        remainder section

until false
```

```
Test-and-Set(target)

    result := target
    target := TRUE
    return result
```

Process 1	Process 2
• wants to set target to TRUE • target is changed to TRUE • result comes back as FALSE so no busy waiting	• wants to set target to TRUE • it receives the result TRUE • busy waits as long as Process 1 is in the critical section
• leaves critical section • sets target to FALSE	

TSL DRY-RUN

Scene-01:

- Process P_0 arrives.
- It executes the test-and-set(Lock) instruction.
- Since lock value is set to 0, so it returns value 0 to the while loop and sets the lock value to 1.
- The returned value 0 breaks the while loop condition.
- Process P_0 enters the critical section and executes.
- Now, even if process P_0 gets preempted in the middle, no other process can enter the critical section.
- Any other process can enter only after process P_0 completes and sets the lock value to 0.

TSL DRY-RUN

- **Scene-02:**
- Another process P_1 arrives.
- It executes the test-and-set(Lock) instruction.
- Since lock value is now 1, so it returns value 1 to the while loop and sets the lock value to 1.
- The returned value 1 does not break the while loop condition.
- The process P_1 is trapped inside an infinite while loop.
- The while loop keeps the process P_1 busy until the lock value becomes 0 and its condition breaks.

TSL DRY-RUN

- **Scene-03:**
- Process P_0 comes out of the critical section and sets the lock value to 0.
- The while loop condition breaks.
- Now, process P_1 waiting for the critical section enters the critical section.
- Now, even if process P_1 gets preempted in the middle, no other process can enter the critical section.
- Any other process can enter only after process P_1 completes and sets the lock value to 0.

Compare And Swap(CAS)

The CAS instruction takes three parameters:

- a location, an "expected value" for that location, and a new value for the location.

- It checks that the contents of the location match the expected value.
  - If it matches, it replaces them with the new value,
  - if it does not match, it has no effect. In any case, the previous value of the variable is returned.

1. Executed atomically
2. Returns the original value of passed parameter **`value`**
3. Set the variable **`value`** the value of the passed parameter **`new_value`** but only if **`*value == expected`** is true. That is, the swap takes place only under this condition.

Compare & swap (CAS)

Compare the value of a memory location with an old value. If they match then replace with a new value

ATOMIC

```
int compare_and_swap(int *x, int old, int new) {
    int save = *x;
    if (save == old)
        *x = new;
    return save;  /* always return location contents */
}
```

How you use it to grab a critical section:
Avoid the race condition – set *locked* to 1 only if *locked* was still set to 0.

```
while (compare_and_swap(&locked, 0, 1) != 0) ;
          /* spin until locked == 0 */
/* if we got here, locked got set to 1 and we have it */
/* do critical section */
locked = 0;   /* release the lock */
```

Mutual Exclusion Using Machine Instructions

Advantages

- Applicable to any number of processes on single or multiple CPUs sharing main memory
- It is simple and therefore easy to verify

Disadvantages

- Busy-waiting consumes processor time
- Starvation is possible
  - A process leaves a CS
  - More than one process is waiting
- Deadlock possible if used in priority-based scheduling systems: Ex. scenario
  - Low priority process has the critical region
  - Higher priority process needs it
  - The higher priority process will obtain the CPU to wait for the critical

Activ
Go to

Busy-waiting

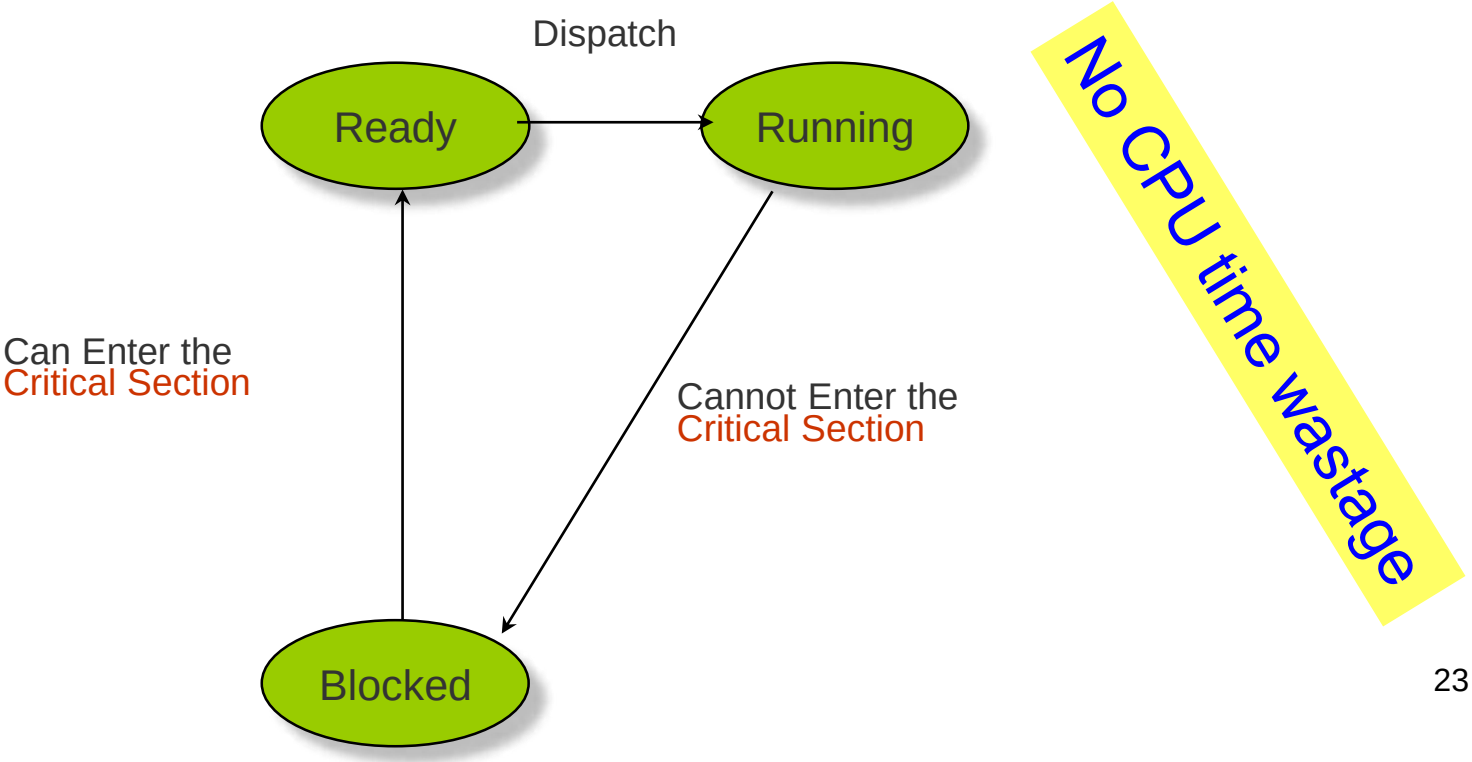
- The problem with this (and S/W based) implementations is busy-waiting.
- What if one thread already has the lock, and another thread wants to acquire the lock?
- The acquiring thread will spin until the thread that already has the lock unlocks it.
- Waste of CPU resources

Sleep and Wakeup

- The solution is that if a process Pi cannot enter the critical section
- Pi should not waste CPU's time by executing the Idle loop.
- Pi's status should be changed from Run to Block
- Pi should be removed from the ready list
- Entered in a block list
- Since, Pi is not in the ready list, no context switch will bring Pi in the run state
- Thus, Pi will not waste CPU's cycle.

Sleep and Wakeup

- Later on, when Pi can enter the critical section, Pi's status should be changed from block to ready
- It should be entered in the ready list.



Sleep and Wakeup: Implementation

- **`Sleep()`** and **`Wakeup()`** can be two system calls.
- When a process Pi determines that it can not enter the critical section it calls **`sleep()`**
  - Pi Entered in the block list
- When the other process finishes with the critical section, it wakesup Pi, **`Wakeup(Pi)`**
  - Pi Entered in the ready list
- Wakeup takes a single input parameter, which is the ID of the process to be unblocked

Priority Inversion

- Consider a task L, with low priority, that requires a resource R.
- Now, consider another task H, with high priority.
  - This task also requires resource R.
  - If H starts *after* L has acquired resource R, then H has to wait to run until L relinquishes resource R.

Priority Inversion

- Problem arises When a new task M (which does not use R) starts with medium priority during this time.

Priority Inversion

- Since R is still in use (by L), H cannot run.
- Since M is the highest priority unblocked task, it will be scheduled before L.
- Since L has been preempted by M, L cannot relinquish R.
- So M will run until it is finished, then L will run - at least up to a point where it can relinquish R - and then H will run.

Priority Inversion

- Thus, in the scenario above, a task with medium priority ran before a task with high priority, effectively giving us a priority inversion.

Possible Solution: Priority inheritance

- If high priority task has to wait for some resource shared with an executing low priority task,
  - the low priority task is temporarily assigned the priority of the highest waiting priority task
  - keeping medium priority tasks from pre-empting the (originally) low priority task, and thereby affecting the waiting high priority task as well.